

Thirteen quick tips to run species distribution models on high-performance computers.

Authors

Co-first: Luca Bütikofer, 1

Co-first: Wim Hordijk, 2

Antoine Adde, 3

Bart Steen, 1

Olivier Brönniman 1,4

Antoine Guisan 1,4

Federico Riva, 2 Email: f.riva@vu.nl ORCID: <https://orcid.org/0000-0002-1724-4293>

1. Department of Ecology and Evolution, University of Lausanne, Biophore building, 1015 Lausanne, Switzerland
2. Department of Environmental Geography, Institute of Environmental Studies, Vrije Universiteit Amsterdam, Amsterdam, the Netherlands
3. Department of Aquatic Ecology, Eawag, Swiss Federal Institute of Aquatic Science and Technology, Dübendorf, Switzerland
4. Institute of Earth Surface Dynamics, University of Lausanne, Geopolis building, 1015 Lausanne, Switzerland

Abstract

Species Distribution Models (SDMs) are increasingly being applied to high-resolution data across large spatial and taxonomic scales, pushing the limits of personal computing. High Performance Computing (HPC) environments offer a powerful solution, but also introduce challenges unfamiliar to many ecologists that rely on SDM workflows. This article provides guidelines to overcoming such challenges. Drawing from direct experience in fitting SDMs for thousands of species across millions of spatial cells, we outline common pitfalls and offer 13 hands-on quick tips, including concrete examples and actionable recommendations. Our goal is to make HPC-based SDMs more accessible, reproducible, sustainable, and efficient for biodiversity researchers.

Introduction

Species distribution models (SDMs) are an important tool in biodiversity research (Guisan, et al., 2013). Historically, analyses based on SDMs have been limited by the coarse resolution of available environmental data. However, the emergence of high-resolution datasets (Jetz, et al., 2019) has shifted the primary limitation from data availability to computational capacity. At the same time, growing evidence indicates that high-resolution data can provide important insights on local distribution patterns (Riva, et al., 2024, Bütikofer, et al., 2020) suggesting that the demand for computing resources will dramatically increase in future SDM applications (Adde et al. 2023). This led us to develop pipelines to fit SDMs on high performance computing (HPC) environments.

HPC environments provide two main advantages compared to personal computers; (1) they offer parallelisation capacity, which allow thousands of spatial units, iterations, replications, or species to be processed simultaneously (i.e., “in parallel”), and (2) they offer substantially larger memory resources, allowing models to load and manipulate high- resolution data entirely in memory. Implementing SDM workflows to HPC environments, however, presents significant technical challenges (Adde et al. 2025). If you are starting from scratch and need to set up a HPC environment, we refer to the Ten Simple Rules laid out by Galiwango *et al.* (2025). Here, we share 13 quick tips specific to implementation of SDM workflows to HPC environments.



Figure 1: Four areas of focus for effective implementation of SDM workflows to HPC environments.

Tip 1: Make friends with the command line

Learning some basic shell commands is a first essential step to fit SDMs on HPCs. HPC clusters generally run the Linux operating system, in contrast to Windows or MacOS, which are commonly used on personal computers. Although Linux typically includes a Graphic User Interface (GUI) on personal computers, HPC clusters do not provide one. Instead, interaction with the operating system takes place through a Command Line Interface (CLI), where actions (e.g., navigating, copying/pasting, writing, saving, and executing) are performed by typing text, typically using shell scripting. Alnasir (2021) shared helpful tips to get you started, but long-format guides to facilitate this process also exist (e.g., Siever, Weber, Figgins, Love, & Robbins (2005)).

Tip 2: Capitalize on remote file management software

Connecting to an HPC environment from a Windows personal computer can be difficult. Users can benefit greatly from third-party software such as MobaXterm (Mobatek, 2025) or equivalent. These applications combine a CLI with a file browser GUI, enabling convenient file management on the HPC system (e.g., uploading, downloading, and copying files). They also include a basic text editor for writing and editing job scripts and other text files directly on the cluster. Note that since macOS is based on a UNIX-like operating system, the built-in Terminal app can be used to access the HPC environment without requiring third-party software. MacOS users who prefer GUI-based file browsing on HPC environments, can install third-party software like FileZilla (FileZilla Project, 2024).

Tip 3: Get to know the Job Scheduler

Running code on a HPC is not as straightforward as running code on your laptop. On a personal computer, biologists typically write and execute code designed to fit SDMs (typically in R or Python software) within an Integrated Development Environment (IDE, e.g., RStudio or PyCharm). However, in an HPC environment, the code is first written and then passed on as a “job” (i.e., self-contained task) to a Job Scheduler for its execution. The Job Scheduler (e.g., SLURM or PBS) manages, queues, and executes the job following scheduling rules. Each job is provided to the job scheduler as a text file with a specific format (see example sbatch file in Appendix), which specifies the software modules to load, the memory and number of processing cores required, the expected time of execution, and the code itself. If processing requires more than the requested resources (typically more time or memory), the job is automatically killed by the system.

Tip 4: Keep your files tidy

Data management is critical for SDM-HPC workflows. File names should be simple, consistent, and machine-readable: we recommend using full species names wherever possible, avoiding spaces, which can break command-line scripts (e.g.

“Taxus_baccata.tif”). We also recommend organizing data using separate folders for different data types, while keeping file names identical across folders (e.g. “Occurrences/Taxus_baccata.csv”), rather than encoding metadata within file names (e.g. “Occurrences_Taxus_baccata.csv”). This folder-based organization improves navigation of large workflows and facilitates matching species across datasets during analysis and visualization. Additionally, SDMs depend on spatially-explicit data that can be easily mishandled resulting in errors (Guisan & Thuiller 2005, Anderson *et al.* 2020). We suggest routinely checking coordinate reference systems, extent, and resolution of all geospatial files, using resampling functions to re-format to consistent spatial standards. Finally, after uploading, we suggest checking file permissions with the Shell command “ls -l”, and grant writing and executing rights when needed with the “chmod” command (see Appendix). This is important because when files and folders are transferred from a personal computer to the HPC file system, they do not always preserve the same permission settings (e.g. read, write, execute), which may cause jobs to fail.

Tip 5: Choose efficient file formats

SDM applications often involve large geospatial datasets such that inefficient file formats can create Input/Output bottlenecks on HPCs, and consume excessive disk space. We suggest preferring formats such as GeoTIFF or NetCDF for both environmental covariate and output SDM maps, as their compression allows for reduced file size. For GeoTIFFs, we recommend using lossless compression options. When possible, output maps should be saved as integers rather than float rasters; for continuous suitability values, consider multiplying the values by 100 and rounding to the nearest integer. This preserves precision to two decimal places while benefiting from the efficiency of integer formats.

Tip 6: Parallelize wisely

HPC environments allow for parallelization at two levels: (1) internally to a job, by requesting the scheduler multiple processing cores and allocating parallel tasks (i.e. “parallel threads”) to them in the provided R or Python script (e.g. with “parallel loops”), or (2) across jobs, by requesting the scheduler to execute a job array, which executes the provided R or Python script many times, each time changing some of the inputs (e.g., which species to “run it on”; see “--array” in “Example sbatch script” in SI). In both cases, we suggest generating a table containing all parameter combinations and have each parallel thread or job process one row of this table. For parallelization internally to a job, configure your scripts to match your requested resources. If you request 4 CPUs (see “--cpus-per-task 4” in “Example sbatch script” in SI), set the same number of parallel threads in your script, (e.g. with `parallel::mclapply(mc.cores = 4)` in R).

Tip 7: Be nimble with memory allocation

Even though HPC environments provide far greater parallelisation capacity and memory resources, these two can come into conflict. Increasing the memory required by a job often forces a reduction in the number of parallel threads that can be allocated. Efficient HPC workflows therefore require explicitly managing the trade-off between (1) the number of parallel processes, and (2) the amount of RAM available to each process (Fig. 2). A possibility to reduce per-process memory requirements is to re-code memory-intensive functions from standard SDM packages. For example, we modified a very useful function from the ENMeval R package (Kass et al., 2021) to avoid loading large raster files entirely into memory (see Appendix), lowering RAM usage per job. When memory demands remain high after code optimization, a straightforward strategy is to run fewer parallel processes, but with higher memory allocated to each. For instance, on a cluster with 128 cores and 256 GB of RAM total, you could run 64 species in parallel—rather than one per core with 2 GB each—increasing the memory allocated to each species to 4 GB, thus preventing memory-related job failures. Finally, for memory-intensive steps such as model projections, memory use can be reduced by splitting computations into data subsets. In our case, we divided the SDM projection area into “tiles” that were processed sequentially, or with limited parallelism, and later merged into a single output map (see Appendix). Together, these examples illustrate a general HPC principle: trading parallelism and additional computing time for reduced memory use, often at the cost of more complex scripts, but with substantially improved robustness and scalability of SDM workflows.

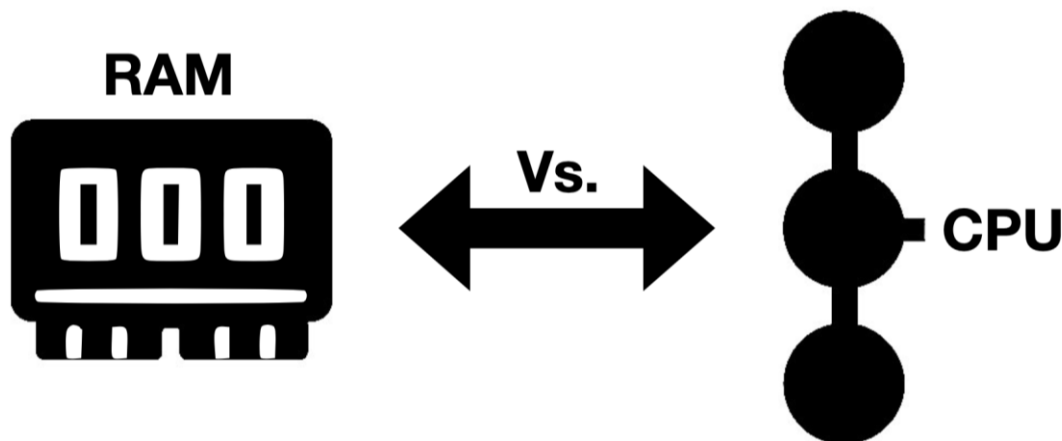


Figure 2: Balancing memory and parallelization is at the core of transitioning scripts to HPC environments.

Tip 8: Clean your workspace

When using R, we recommend setting an efficient garbage collection strategy to free unused memory throughout the job execution. Normally, R calls the garbage collection routine based on the proportion of total memory available. Therefore, garbage collection is relaxed on a node with, e.g., 256GB of memory. This can cause issues with parallel calculations. R can be forced to use a more aggressive garbage collection strategy by setting the R environment variable `R_GC_MEM_GROW` to zero by placing a file named `".Renvi ron"` in either your home folder or the scratch folder from where the compute jobs are submitted to the job scheduler, with the following line in it:

```
R_GC_MEM_GROW=0
```

This file is then automatically read by R at startup. We have achieved up to a 25% reduction in memory use on the HPC environment just with this simple step.

Tip 9: Partner with your HPC admin team

Many R or Python packages to run SDMs (Kass et al. 2024) rely on dependencies (e.g. GDAL or proj) which may not be available on your cluster and can only be installed by the administration team. Without these dependencies, code that works on your computer will likely crash when executed on an HPC environment—or code that works on one HPC might fail on another. Make sure to liaise with your HPC admin team to have all required dependencies installed before running your scripts. Also, remember to load them in your sbatch script (e.g. with the `"module load"` SLURM command; see Appendix).

Tip 10: Use interactive sessions

Most HPC clusters allow connecting to a node and using it interactively, just like if you were running a command line terminal on your laptop. This "interactive mode" is designed for testing your code before submitting it as a standard, non-interactive job to the Job Scheduler. This way, a script can be run line-by-line to see where errors might occur. IDEs like RStudio or PyCharm allow users to send individual commands with keyboard shortcuts either to the console (ctrl + Enter) or to a command line terminal (ctrl + Shift + Enter). When working locally, users usually rely on an IDE and test their code via the IDE console. On an HPC system, the same IDE can be used, but code should instead be executed from a terminal window within the IDE that is connected to the cluster (e.g. via SSH). When combined with an interactive session on a compute node, this setup allows users to test and debug R or Python code on the HPC with only minor differences compared to running it locally.

Tip 11: Capitalize on different file storage systems

HPC environments generally rely on two main file storage systems: a short-term, low-latency file system, intended for temporary data and active computations, and a long-

term but slower file system, designed for durable storage and archiving. The long-term storage system, generally referred to as the “home” or “work” directory, is typically permanent, regularly backed-up, but with slower file access from the cluster CPUs. The short-term storage system, generally referred to as the “scratch” directory, has faster read/write speed and larger capacity, but it is not automatically backed-up and is regularly purged. We suggest that a typical SDM-HCP workflow should leverage this dual file system. It should involve storing code and large input files on the long-term storage system (e.g., scripts, environmental variables, species occurrence data). For the active jobs, input files can be transferred to the scratch system, where temporary intermediate files (e.g., model objects) are also saved. Lastly, the final results are saved back to the long-term storage.

Tip 12: Measure twice, submit once

Overestimating memory or CPU requirements leads to inefficient runs on the HPC environments and places unnecessary load on shared resources. Yet, underestimating memory or CPU causes job failures. We suggest estimating resource requirements using small pilot jobs, which can be done, e.g., by focusing on a single species at full spatial resolution and extent, and monitoring memory, CPU, and runtime usage, before scaling-up to the full analysis. Use SLURM commands (e.g., `sacct`, `sstat`, or `seff`) to track resource usage and adjust your job requests accordingly. Testing is key to monitor efficiency during computationally-intensive jobs. Some clusters provide live dashboards or commands to view per-job memory and CPU use. For example, SLURM users can monitor jobs in real time with “`sstat`” (showing memory and CPU consumption of a running job) or “`scontrol show job <job_id>`”. Many institutions also offer web-based monitoring dashboards (such as Open OnDemand) where users can check job status, resource usage, and queue positions “live”.

Tip 13: Consider the planet

Tips 6, 9 and 12 serve another fundamental purpose: reducing carbon emissions associated with HPC usage. The two main causes of energy waste and excessive CO₂ output are memory overallocation (Lannelongue et al. 2021) and premature cancellation of cluster jobs. Both can often be avoided by running interactive sessions (tip 9) and by using resources monitoring software (tip 12). Attuning code in interactive sessions helps identify coding errors, thus avoiding unnecessary job failures, and the resources monitoring indicates the power consumption of the session after its conclusion. Additionally, employing strategically placed “flags” helps isolate RAM-intensive portions of the code. Finally, if Large Language Models have been installed on the HPC cluster or used to improve the cluster scripts, they can be chatted to in interactive sessions and used as coding assistants, which is safer and consumes less electricity than their online counterparts (Ding et al. 2025).

Conclusions

SDMs fitted for millions of pixels and thousands of species will be necessary to answer cutting-edge questions in environmental and biodiversity sciences. Such models are increasingly implementable thanks to advances in data availability and computing infrastructure—especially HPC environments. However, harnessing HPC environments requires technical skills that are not typically required for local computing. If your institution provides access to HPC resources, it most likely also provides some introductory course to learn the properties of your HPC environment (e.g., file management system, job scheduling, available computational resources, etc.) and how to use them. In our experience, spending a few hours learning the specifics of a HPC environment can save you days of debugging.

More broadly, our experience highlights that many common issues can be addressed through relatively straightforward practices, which we summarized in this Quick Tips paper. With preparation and awareness, ecologists can unlock the full potential of HPC environments to accelerate SDM workflows and scale up biodiversity assessments. Yet, we also advise that ecologists should aim at running such analyses in an energetically parsimonious way, reducing our contribution to the biodiversity declines we are documenting with SDMs. We hope this paper helps others avoid the common pitfalls we faced as we developed SDM workflows for HPC environments, accelerating the broader adoption of best practices in computational biology.

Acknowledgements

We acknowledge funding support from Swiss Re and enabling partners AXA Research Fund, WWF, Swiss Re Institute and as a service contributor, EY for the "Modelling biodiversity and ecosystem service loss to advance resilience" project, the Swiss Confederation through the [Valpar.ch](https://www.valpar.ch) project, and the European Commission through the COUSIN Horizon project.

References

- Adde, A., Rey, P.-L., Külling, N. *et al.* *SDMapCH*: a Comprehensive database of >7,500 modelled species habitat suitability maps for Switzerland. *Sci Data* 12, 1752 (2025). <https://doi.org/10.1038/s41597-025-06037-x>
- Adde, A., Külling, N., Rey, P.-L., Fopp, F., Brun, P., Broenniman, O., Lehmann, A., Petitpierre, B., Zimmermann, N. E., Pellissier, L., Altermatt, F. & Guisan, A. (2024) Projecting Untruncated Climate Change Effects on Species' Climate Suitability: Insights From an Alpine Country. *Global Change Biology*, 30.

265 Adde, A., Rey, P. L., Brun, P., Kuelling, N., Fopp, F., Altermatt, F., Broennimann, O.,
 266 Lehmann, A., Petitpierre, B., Zimmermann, N. E., Pellissier, L. & Guisan, A. (2023) N-
 267 SDM : a high-performance computing pipeline for Nested Species Distribution
 268 Modelling. *Ecography*, 2023(6):e06540.

269 Alnasir, J.J. (2021) Fifteen quick tips for success with HPC, i.e., responsibly BASHing
 270 that Linux cluster. *PLoS Comput Biol* 17(8): e1009207.

271 Anderson, R. P., Araujo, M. B., Guisan, A., Lobo, J. M., Martinez-Meyer, E., Peterson, A.
 272 T. & Soberon, J. M. (2020) Optimizing biodiversity informatics to improve information
 273 flow, data quality, and utility for science and society. *Frontiers of Biogeography*, 12.

274 Bütikofer L, Anderson K, Bebbber D P, Bennie J J, Early R I and Maclean I M D 2020 The
 275 problem of scale in predicting biological responses to climate *Global Change Biology*
 276 26 6657–66

277 Ding, Z., Wang, J., Song, Y., Zheng, X., He, G., Chen, X., Zhang, T., Lee, W.-J., & Song, J.
 278 (2025). Tracking the carbon footprint of global generative artificial intelligence. *The*
 279 *Innovation*, 6(5), 100866.

280 FileZilla. FileZilla Project, Version 3.66, 2024, <https://filezilla-project.org>.

281 Galiwango R., Whalen C.J., Kebirungi G., Atwine M.T., Kimera R., Ssekagiri A., et al.
 282 (2025) Ten simple rules for building and maintaining sustainable high-performance
 283 computing infrastructure for research in resource-limited settings. *PLoS Comput Biol*
 284 21(9): e1013481. <https://doi.org/10.1371/journal.pcbi.1013481>

285 Guisan, A., Tingley, R., Baumgartner, J. B., Naujokaitis-Lewis, I., Sutcliffe, P. R., Tulloch,
 286 A. I., . . . Buckley, Y. M. (2013). Predicting species distributions for conservation
 287 decisions. *Ecology Letters*, 16(12), 1424-1435.

288 Guisan, Antoine, and Wilfried Thuiller. "Predicting species distribution: offering more
 289 than simple habitat models." *Ecology letters* 8.9 (2005): 993-1009.

290 Jetz, W., McGeoch, M. A., Guralnick, R., Ferrier, S., Beck, J., Costello, M. J., . . . Turak, E.
 291 (2019). Essential biodiversity variables for mapping and monitoring species
 292 populations. *Nature Ecology & Evolution*, 3, 539-551.

293 Kass, J. M., Smith, A. B., Warren, D. L., Vignali, S., Schmitt, S., Aiello-Lammens, M. E.,
 294 Arlé, E., Barbosa, A. M., Broennimann, O., Cobos, M. E., Gueguen, M., Guisan, A.,
 295 Merow, C., Naimi, B., Nobis, M. P., Ondo, I., Osorio-Olvera, L., Owens, H. L., Pinilla-
 296 Buitrago, G. E., Sanchez-Tapia, A., Thuiller, W., Valavi, R., Velazco, S. J. E., Zizka, A. &
 297 Zurell, D. (2024) Achieving higher standards in species distribution modeling by
 298 leveraging the diversity of available software. *Ecography*.

299 Kass, J. M., Muscarella, R., Galante, P. J., Bohl, C. L., Pinilla-Buitrago, G. E., Boria, R. A.,
 300 . . . Anderson, R. P. (2021). ENMeval 2.0: Redesigned for customizable and reproducible

301 modeling of species' niches and distributions. *Methods in Ecology and Evolution*, 12(9),
302 1602-1608.

303 Kays, R., McShea, W. J. & Wikelski, M. (2020) Born-digital biodiversity data: Millions and
304 billions. *Diversity and Distributions*, 26, 644-648.

305 Lannelongue, L., Grealey, J., & Inouye, M. (2021). Green Algorithms: Quantifying the
306 Carbon Footprint of Computation. *Advanced Science*, 8(12), 2100707.
307 <https://doi.org/10.1002/advs.202100707>

308 Mobatek. (2025). MobaXterm. Retrieved from <https://mobaxterm.mobatek.net>

309 Phillips, S. J., Anderson, R. P., Dudik, M., Schapire, R. E., & Blair, M. E. (2017). Opening
310 the black box: an open-source release of Maxent. *Ecography*, 40(7), 887-893.

311 Riva, F., Martin, C. J., Acedo, C. G., Bellon, E. N., Keil, P., Moran-Ordóñez, A., . . .
312 Guisan, A. (2024). Incorporating effects of habitat patches into species distribution
313 models. *Journal of Ecology*, 112(10), 2162-2182.

314 Siever, E., Weber, A., Figgins, S., Love, R., & Robbins, A. (2005). *Linux in a Nutshell* (Fifth
315 ed.). O'Reilly.